bfs

```python
from collections import deque

def bfs(graph : dict, root : int, visited : set[int]):
    q = deque()
    q.append(root)
    visited.add(root)
    while q:
        v = q.popleft()
        for n in graph[v]:
            if n not in visited:
                visited.add(n)
                q.append(n)
```

binary_search

```python
def binary_search(arr : list[int], target: int) -> int:
    lo = 0
    hi = len(arr) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if target < arr[mid]:
            hi = mid - 1
        elif target > arr[mid]:
            lo = mid + 1
        else:
            return mid
    return lo #Correct position if not found
```

cart_tree

```python
import random

class Treap:
    def __init__(self, key, ortho):
        self.key = key              # x / index
        self.priority = ortho       # value stored (max-heap here)
        self.left = self.right = None

def rotate_left(p):
    r = p.right
    p.right = r.left
    r.left = p
    return r

def rotate_right(p):
    l = p.left
    p.left = l.right
    l.right = p
    return l

def treapify(root):
    if root.left and root.left.priority > root.priority:
        return rotate_right(root)
    if root.right and root.right.priority > root.priority:
        return rotate_left(root)
    return root

def insert(root, key, ortho):
    if not root:
        return Treap(key, ortho)
    if key < root.key:
        root.left = insert(root.left, key, ortho)
    else:
        root.right = insert(root.right, key, ortho)
    return treapify(root)

def delete(root, key):
    if not root:
        return None

    if key < root.key:
        root.left = delete(root.left, key)
    elif key > root.key:
        root.right = delete(root.right, key)
    else:
        # found node
        if not root.left:
            return root.right
        if not root.right:
            return root.left

        # both children: rotate higher-priority child up
        if root.left.priority > root.right.priority:
            root = rotate_right(root)
            root.right = delete(root.right, key)
        else:
            root = rotate_left(root)
            root.left = delete(root.left, key)

    return root

def range_search(root, L, R):
    """
    Return node with maximal priority whose key is in [L, R].

    Uses Cartesian-tree / RMQ invariant:
    - If [L, R] is left of root.key: go left.
    - If [L, R] is right of root.key: go right.
    - Else: root.key in [L, R] and root is the max in that range.
    """
    node = root
    while node:
        if L <= node.key <= R:
            return node
        if R < node.key:
```

```
            node = node.left
        elif L > node.key:
            node = node.right
    return None

def print_tree(node, indent=0):
    if node:
        print_tree(node.right, indent + 4)
        print(" " * indent + f"{node.key} (prio={node.priority})")
        print_tree(node.left, indent + 4)

if __name__ == "__main__":
    # Example: keys = indices, priority = value (max-heap RMQ)
    vals = [5, 1, 9, 3, 7, 2, 8]
    root = None
    for i, v in enumerate(vals):
        root = insert(root, i, v)

    print("Tree:")
    print_tree(root)

    L, R = 3, 5
    res = range_search(root, L, R)
    if res:
        print(f"\nRange [{L}, {R}] -> index {res.key}, value {res.priority}")
    else:
        print(f"\nRange [{L}, {R}] -> no result")
```

dfs

```python
def dfs_recursive(graph : dict, root : int, visited : set[int]):
    visited.add(root)
    for n in graph[root]:
        if n not in visited:
            dfs_recursive(graph, n, visited)

def dfs_iterative(graph : dict, root: int, visited : set[int]):
    s = [root]
    while s:
        v = s.pop()
        if v not in visited:
            visited.add(v)
            for n in graph[v]:
                s.append(n)
```

gaussian_elimination

```python
# https://www.geeksforgeeks.org/dsa/gaussian-elimination/
MAXN = 100

def partial_pivot(A, n):
    for i in range(n):
        pivot_row = i
        for j in range(i + 1, n):
            if abs(A[j][i]) > abs(A[pivot_row][i]):
                pivot_row = j
        if pivot_row != i:
            A[i], A[pivot_row] = A[pivot_row], A[i]
        for j in range(i + 1, n):
            factor = A[j][i] / A[i][i]
            for k in range(n + 1):
                A[j][k] -= factor * A[i][k]

def back_substitute(A, n):
    x = [0]*n
    for i in reversed(range(n)):
        sum_val = sum(A[i][j] * x[j] for j in range(i + 1, n))
        x[i] = (A[i][n] - sum_val) / A[i][i]
    return x

if __name__ == "__main__":
    n = 3
    A = [[3.0, 2.0, -4.0, 3.0],
         [2.0, 3.0, 3.0, 15.0],
         [5.0, -3, 1.0, 14.0]]
    partial_pivot(A, n)
    x = back_substitute(A, n)
    print("Solution for the system:")
    for i in range(n):
        print(f"{x[i]:.6f}")

# 3.000000
# 1.000000
# 2.000000
```

mst

```python
from heapq import heappush, heappop

def boruvkas(n, edges):
    m = dict()
    p = list(range(n))
    mst = []
    def f(x):
        if p[x] != x: p[x] = f(p[x])
        return p[x]
    while len(mst) < n - 1:
        m.clear()
        for u, v, w in edges:
            ru, rv = map(f, (u, v))
            if ru != rv:
                if ru not in m or w < m[ru][2]: m[ru] = (u, v, w)
                if rv not in m or w < m[rv][2]: m[rv] = (u, v, w)
        for e in m.values():
            if f(e[0]) != f(e[1]):
                mst.append(e)
                p[f(e[0])] = f(e[1])
        if not m: break
    return mst

def prims(n, edges):
    g = [[] for _ in range(n)]
    for u, v, w in edges:
        g[u].append((w, u, v))
        g[v].append((w, v, u))
    start = 0
    for i in range(n):
        if g[i]:
            start = i
            break
    vis, h, r = {start}, list(g[start]), []
    while h and len(r) < n - 1:
        w, u, v = heappop(h)
        if v not in vis:
            vis.add(v)
            r.append((u, v, w))
            for edge in g[v]:
                if edge[2] not in vis:
                    heappush(h, edge)
    return r

def kruskals(n, edges):
    p = list(range(n))
    def f(x):
        if p[x] != x: p[x] = f(p[x])
        return p[x]
    r = []
    for u, v, w in sorted(edges, key=lambda e: e[2]):
        ru, rv = f(u), f(v)
        if ru != rv:
            p[ru] = rv
            r.append((u, v, w))
            if len(r) == n - 1: break
    return r

def main():
    import time
    inp = ""
    with open("../test_data/graph_input.txt", encoding="utf-8") as f:
        inp = f.read()
    edges = [tuple(map(int, line.split(','))) for line in inp.strip().split('\n')]
    n = max(max(u, v) for u, v, _ in edges) + 1
    algorithms = (("Boruvka's", boruvkas), ("Prim's", prims), ("Kruskal's", kruskals))
    reference_weight = sum(w for *_, w in algorithms[0][1](n, edges))
    for name, alg in algorithms:
        start = time.perf_counter()
        result = alg(n, edges)
        elapsed = time.perf_counter() - start
        total_weight = sum(w for *_, w in result)
        status = "PASSED" if total_weight == reference_weight and len(result) == n - 1 else "FAILED"
        print(f"{name:12} {status:6} | Time: {elapsed*1000:7.3f}ms | Weight: {total_weight:6} | Edges:
```

```
{len(result)}")

if __name__ == "__main__":
    main()
```

treap

```python
import random

class Treap:
    def __init__(self, key):
        self.key = key
        self.priority = random.getrandbits(64)
        self.left = self.right = None

def rotate_left(p):
    r = p.right
    p.right = r.left
    r.left = p
    return r

def rotate_right(p):
    l = p.left
    p.left = l.right
    l.right = p
    return l

def treapify(root):
    if root.left and root.left.priority > root.priority:
        return rotate_right(root)
    if root.right and root.right.priority > root.priority:
        return rotate_left(root)
    return root

def insert(root, key):
    if not root:
        return Treap(key)
    if key < root.key:
        root.left = insert(root.left, key)
    else:
        root.right = insert(root.right, key)
    return treapify(root)

def delete(root, key):
    if not root:
        return None

    if key < root.key:
        root.left = delete(root.left, key)
    elif key > root.key:
        root.right = delete(root.right, key)
    else:
        # Found node to delete
        if not root.left:
            return root.right
        if not root.right:
            return root.left

        # Both children exist: rotate the higher-priority child up
        if root.left.priority > root.right.priority:
            root = rotate_right(root)
            root.right = delete(root.right, key)
        else:
            root = rotate_left(root)
            root.left = delete(root.left, key)

    return root

def search(node, key):
    if not node or key == node.key:
        return node
    return search(node.left if key < node.key else node.right, key)

def print_tree(node, indent=0):
    if node:
        print_tree(node.right, indent + 4)
        print(" " * indent + f"{node.key} (p={node.priority})")
        print_tree(node.left, indent + 4)

if __name__ == "__main__":
    random.seed(42)
    vals = random.sample(range(100), 20)
```

```
root = None
for v in vals:
    root = insert(root, v)
print("Initial tree:")
print_tree(root)
q = vals[5]
print(f"\nSearching for {q}:")
if found := search(root, q):
    print(f"Found {found.key} (p={found.priority})")
else:
    print("Not found")
print(f"\nDeleting {q}:")
root = delete(root, q)
print_tree(root)
```